

Shapley Values

1 Shapley Values for Cooperative Games

Consider a **coalitional game** with player set N where any subset (**coalition**) $S \subseteq N$ can form and achieve value given by a **characteristic function** (also called a **value function**) $v : 2^N \rightarrow \mathbb{R}$ with $v(\emptyset) = 0$. In the machine learning context we will refer to v as the **value function**. The problem is to find a fair **allocation** $\phi_i(v)$ for each player $i \in N$ that distributes $v(N)$ according to agreed fairness principles.

Four axioms define any fair **allocation mechanism**:

1. **Efficiency:** The axiom requires $\sum_{i \in N} \phi_i(v) = v(N)$. It ensures the full value of the player set is distributed.
2. **Symmetry:** The axiom states that if $v(S \cup \{i\}) = v(S \cup \{j\})$ for all S not containing i or j , then $\phi_i(v) = \phi_j(v)$. It enforces equal treatment of players with identical marginal contributions.
3. **Dummy Player:** The axiom states that if $v(S \cup \{i\}) = v(S)$ for all S not containing i , then $\phi_i(v) = 0$. It recognises that players who never contribute value receive none of the payoff.
4. **Linearity:** The axiom requires $\phi_i(av + bw) = a\phi_i(v) + b\phi_i(w)$ for any value functions v, w and scalars $a, b \in \mathbb{R}$. It ensures additivity across games and positive homogeneity with respect to scaling.

These four axioms uniquely determine the allocation. The **Shapley value** for player i is given by:

$$\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} [v(S \cup \{i\}) - v(S)] \quad (1)$$

Derivation of the weight $\frac{|S|!(|N| - |S| - 1)!}{|N|!}$: Consider all $|N|!$ possible arrival orders (**permutations**) of the N players. For player i to join exactly the coalition S , the players in S must arrive before i , and the remaining $|N| - |S| - 1$ players must arrive after i . There are $|S|!$ ways to order the players arriving before i , and $(|N| - |S| - 1)!$ ways to order those arriving after i . Thus, the number of permutations where i joins coalition S is $|S|!(|N| - |S| - 1)!$, and the

probability is $\frac{|S|!(|N|-|S|-1)!}{|N|!}$. Therefore the Shapley value equals the expected marginal contribution under a uniform random arrival order.

The term $v(S \cup \{i\}) - v(S)$ is player i 's **marginal contribution** when joining coalition S .

Equivalently, averaging over all **permutations** $\pi \in \Pi(N)$, where $\Pi(N)$ denotes the set of all permutations of N :

$$\phi_i(v) = \frac{1}{|N|!} \sum_{\pi \in \Pi(N)} [v(S_\pi^i \cup \{i\}) - v(S_\pi^i)] \quad (2)$$

where S_π^i is the set of players appearing before i in permutation π .

2 Machine Learning Feature Attribution as Shapley Values

2.1 Formal Setup

We now set up the general SHAP framework for feature attribution. Consider a **predictive model** $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and an input $x \in \mathbb{R}^n$ with prediction $f(x)$. The goal is to quantify how much each feature in x contributes to this prediction. We formulate this task as a Shapley value computation by treating each **original feature as a player** in a cooperative game. We index the features by the player set $N = \{1, \dots, n\}$, with feature i corresponding to coordinate x_i .

To connect the model f with coalitional games we introduce the **masked predictor** $f(x; S)$. For any subset (**coalition**) $S \subseteq N$ we interpret S as the set of features that are treated as known, and we define $f(x; S)$ to be the model prediction when only the coordinates in S are available and the remaining coordinates $N \setminus S$ are treated as missing. Notice that we have not yet specified how to compute $f(x; S)$ in practice. This is a **major modeling choice** when computing Shapley values, and we will discuss it in detail in the next section. The special case $f(x; \emptyset)$ is the prediction when no feature information is used, and we refer to this quantity as the **baseline prediction**. We therefore set

$$b = f(x; \emptyset).$$

Given this masked predictor we define the **value function** $v : 2^N \rightarrow \mathbb{R}$ by

$$v(S) = f(x; S) - f(x; \emptyset)$$

for every coalition $S \subseteq N$. By construction, $v(\emptyset) = 0$ and $v(N) = f(x) - f(x; \emptyset)$, so the total game value equals the difference between the actual prediction and the baseline prediction.

A **Shapley explanation** of f at input x is a vector of feature attributions $\phi(x) = (\phi_1(x), \dots, \phi_n(x))$ associated with the game (N, v) . The SHAP framework enforces the **efficiency** condition

$$v(N) = f(x) - f(x; \emptyset) = \sum_{i=1}^n \phi_i(x),$$

so that the total attribution exactly matches the game value $v(N)$. Each attribution $\phi_i(x)$ is defined to be the Shapley value of feature i in the cooperative game (N, v) . Formally, writing $n = |N|$, the Shapley value for feature i is

$$\phi_i(x) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! (n - |S| - 1)!}{n!} [v(S \cup \{i\}) - v(S)].$$

This expression averages feature i 's **marginal contribution** $v(S \cup \{i\}) - v(S)$ over all coalitions S that do not contain i , with weights corresponding to the fraction of permutations in which S precedes i . Equivalently, let $\Pi(N)$ denote the set of all permutations of N and let S_π^i be the set of features that appear before i in permutation π . Then we can write

$$\phi_i(x) = \frac{1}{n!} \sum_{\pi \in \Pi(N)} [v(S_\pi^i \cup \{i\}) - v(S_\pi^i)]$$

and, using the definition of v , this becomes

$$\phi_i(x) = \frac{1}{n!} \sum_{\pi \in \Pi(N)} [f(x; S_\pi^i \cup \{i\}) - f(x; S_\pi^i)].$$

Thus, in the SHAP setting the Shapley values admit two complementary interpretations. First, they **allocate the prediction above the baseline** $f(x; \emptyset)$ so that the attributions sum to $f(x) - f(x; \emptyset)$. Second, each $\phi_i(x)$ equals the **expected marginal change in prediction** when feature i is added to the set of previously included features in a uniformly random ordering.

2.2 Intuition: Toy House Price Example

Suppose we observe five houses, each described by three features—lot size (acres), total square footage, and number of rooms—and a sale price y in dollars, as shown in Table 1.

House	Acres	Square feet	Rooms	Price (\$)
1	0.10	1200	3	220,000
2	0.15	1400	3	260,000
3	0.20	1600	4	300,000
4	0.25	1800	4	340,000
5	0.30	2000	5	380,000

Table 1: Toy dataset of five houses with simple features and sale prices.

Let $x \in \mathbb{R}^3$ collect the three features for a house and let $f(x)$ be a trained regression model's predicted sale price. For a particular house x , for example $x_5 = (0.30, 2000, 5)$, the feature attribution problem is to determine how each coordinate of x (acres, square footage, rooms) contributes to the prediction

$f(x)$. Assume for concreteness that the model outputs $f(x_5) = 360,000$ dollars for house 5.

In the notation of the formal setup, we consider the extended prediction operator $f(x_5; S)$ for coalitions $S \subseteq N = \{1, 2, 3\}$, where the three features correspond to acres, square footage, and rooms. We define the **baseline prediction** to be $f(x_5; \emptyset)$, which in this example we take to be the average sale price across the five houses. If the average price is 300,000 dollars, then $f(x_5) - f(x_5; \emptyset) = 60,000$ dollars. This quantity is the total value $v(N)$ that must be allocated across the three features by the Shapley attributions. The feature attributions decompose the 60,000-dollar improvement above the baseline. Equivalently, we can interpret each $\phi_i(x_5)$ as the expected marginal change in the prediction when feature i is added to a coalition of previously included features in a uniformly random order. Concretely, we start from the baseline prediction $f(x_5; \emptyset)$, choose a random ordering of the three features, and then add them one at a time, recording the increment $f(x_5; S \cup \{i\}) - f(x_5; S)$ each time feature i is added to a set S of features that appear earlier in the ordering. The Shapley value $\phi_i(x_5)$ is the average of this marginal change across all six possible orderings.

To make this explicit, label acres, square footage, and rooms as features 1, 2, 3 so that $N = \{1, 2, 3\}$. For feature 1, the six permutations of N are

$$(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2), (3, 2, 1).$$

In each permutation π , let S_π^1 be the set of features that appear before 1. The marginal contribution of feature 1 in permutation π is

$$\Delta_1^\pi = f(x_5; S_\pi^1 \cup \{1\}) - f(x_5; S_\pi^1).$$

For example, in permutation $(1, 2, 3)$ we have $S_{(1,2,3)}^1 = \emptyset$ and

$$\Delta_1^{(1,2,3)} = f(x_5; \{1\}) - f(x_5; \emptyset),$$

while in permutation $(2, 1, 3)$ we have $S_{(2,1,3)}^1 = \{2\}$ and

$$\Delta_1^{(2,1,3)} = f(x_5; \{1, 2\}) - f(x_5; \{2\}).$$

The Shapley value for feature 1 is then

$$\phi_1(x_5) = \frac{1}{6} \sum_{\pi \in \Pi(\{1,2,3\})} \Delta_1^\pi.$$

Analogous expressions hold for features 2 and 3, obtained by averaging their marginal contributions over the same six permutations.

3 Defining the Masked Predictor

In the previous section, we defined the value function $v(S) = f(x; S) - f(x; \emptyset)$ and introduced the **baseline prediction** $f(x; \emptyset)$. We did not yet specify how

the **masked predictions** $f(x; S)$ are computed. The predictive model f is usually defined only on full feature vectors x , so we must decide how to evaluate the **masked predictor** $f(x; S)$ when only a subset of features is treated as known.

Shapley explanations depend critically on how we define the masked predictor because it is central to the value function:

1. **Baseline prediction:** For a fixed input x , the baseline prediction $f(x; \emptyset)$ determines the total quantity $f(x) - f(x; \emptyset)$ that the Shapley values $\phi_i(x)$ must allocate. The value function $v(S) = f(x; S) - f(x; \emptyset)$ depends directly on the chosen baseline.
2. **Masked predictor semantics:** The masked predictions $f(x; S)$ specify how this total value is distributed across coalitions S . Different definitions of $f(x; S)$ induce different value functions $v(S)$ and thus different Shapley explanations, even when the underlying model f and the input x are fixed.
3. **Faithfulness to the original model:** The Shapley game is built on the masked predictor $S \mapsto f(x; S)$ rather than directly on the original predictor $x \mapsto f(x)$. If $f(x; S)$ is defined using unrealistic hybrids or an inappropriate background distribution, the resulting value function $v(S)$ can behave very differently from f on typical inputs, and the Shapley attributions may fail to faithfully reflect how f actually uses its features.

Several standard choices exist for defining $f(x; S)$. Intuitively, the **masked predictor** $f(x; S)$ represents the prediction when features in S are treated as known and features in $N \setminus S$ are treated as unknown. We evaluate $f(x; S)$ by applying f to a full feature vector in which coordinates in S are fixed to x_S and coordinates in $N \setminus S$ are filled in according to a chosen rule. Several standard choices for this rule appear in the SHAP literature:

1. **Fixed baseline substitution:** Choose a baseline input $x^{(0)} \in \mathbb{R}^n$ (for example, a zero vector or a reference input) and define a hybrid input that uses x_S on coordinates in S and $x_{N \setminus S}^{(0)}$ on coordinates in $N \setminus S$. The masked predictor is $f(x; S) = f(x_S, x_{N \setminus S}^{(0)})$. Unknown features are always replaced with fixed baseline values, which is simple but can evaluate f on out-of-distribution inputs. Formally, for each coalition $S \subseteq N$ we define a hybrid input $x^{(S)} \in \mathbb{R}^n$ by

$$x_i^{(S)} = \begin{cases} x_i, & i \in S, \\ x_i^{(0)}, & i \notin S, \end{cases}$$

so that $x^{(S)} = (x_S, x_{N \setminus S}^{(0)})$ and

$$f(x; S) := f(x^{(S)}).$$

In words, $x^{(S)}$ agrees with the original input x on coordinates in S and with the baseline input $x^{(0)}$ on coordinates in $N \setminus S$.

2. **Mean imputation:** Let $\bar{x}_{N \setminus S} = \mathbb{E}[X_{N \setminus S}]$ denote the mean of the unknown features under the training distribution. The masked predictor is $f(x; S) = f(x_S, \bar{x}_{N \setminus S})$. Unknown features are replaced with their mean training values, which approximates averaging over the marginal distribution using only its first moment.
3. **Marginal expectation (interventional semantics):** Let $P(X)$ denote the joint training distribution over features. The masked predictor is defined by the *marginal expectation*

$$f(x; S) = \mathbb{E}_{X_{N \setminus S} \sim P(X_{N \setminus S})} [f(x_S, X_{N \setminus S})].$$

Unknown features are sampled from their marginal distribution, so conditioning on x_S does not affect their distribution. In practice, this expectation is approximated using samples from the empirical training distribution.

4. **Conditional expectation (conditional semantics):** Instead of breaking feature dependencies, we can preserve them by averaging over the conditional distribution of the unknown features given the known ones. The masked predictor is

$$f(x; S) = \mathbb{E}_{X_{N \setminus S} \sim P(X_{N \setminus S} | X_S = x_S)} [f(x_S, X_{N \setminus S})].$$

Unknown features follow their conditional distribution given $X_S = x_S$, which properly accounts for feature correlations but requires modeling $P(X_{N \setminus S} | X_S)$ in high dimensions.

From a prediction point of view, the **conditional expectation** definition yields the most faithful notion of prediction when only a subset of features is known. Let X denote the random input vector with distribution $P(X)$ and consider predicting the scalar response $f(X)$ under squared error loss when only X_S is observed. The prediction that minimises the expected squared error given $X_S = x_S$ is

$$g_S(x_S) = \mathbb{E}[f(X) | X_S = x_S],$$

which is exactly the conditional masked predictor obtained by averaging $f(x_S, X_{N \setminus S})$ over the conditional distribution of the unknown coordinates $X_{N \setminus S}$ given $X_S = x_S$. In words, conditional expectation computes the best mean-squared-error prediction for $f(X)$ using all information in the observed features X_S while sampling the unknown features in a way that respects their empirical dependencies with X_S . By contrast, marginal expectation ignores these dependencies by treating $X_{N \setminus S}$ as independent of X_S , and fixed-baseline or mean imputation correspond to degenerate choices of $P(X_{N \setminus S} | X_S)$ that are no longer optimal given X_S in this squared-error sense. Thus, when the goal is to explain predictions under the observed data distribution and to treat known and unknown coordinates coherently, conditional expectation provides a principled notion of “best prediction given x_S ” for defining the masked predictor.

This perspective also clarifies why simple **zero** or **mean** baselines can produce unfaithful Shapley games. A zero baseline sets $x^{(0)}$ to the all-zero vector and evaluates hybrids $f(x_S, x_{N \setminus S}^{(0)})$ that may lie far outside the support of the training distribution, so the value function $v(S)$ reflects model behaviour on highly unrealistic inputs rather than on typical data. Similarly, a mean baseline or mean imputation uses a single fixed vector or the marginal mean $\bar{x}_{N \setminus S}$ for all coalitions S , which discards the dependence between X_S and $X_{N \setminus S}$ and does not approximate the conditional expectation $\mathbb{E}[f(X) \mid X_S = x_S]$. In both cases, the masked predictor $S \mapsto f(x; S)$ is anchored to off-manifold or weakly informative reference points, so the induced value function $v(S)$ can deviate substantially from how f behaves on realistic inputs. As a result, Shapley attributions built on zero or mean baselines may emphasise artefacts of the chosen reference rather than faithfully capturing how the original model uses its features across the data distribution.

4 Computational Challenges

Computing exact Shapley values faces two fundamental **computational bottlenecks**:

4.1 Value Function Evaluation

Each evaluation of $v(S)$ requires computing the model’s expected output when only features in S are known. For **conditional expectation**, this requires estimating $P(X_{N \setminus S} \mid X_S = x_S)$, which becomes intractable for high-dimensional features or complex dependencies. For **marginal expectation**, each $v(S)$ evaluation requires multiple model forward passes to average over sampled values of unknown features. With n features, there are 2^n possible coalitions S , each requiring potentially expensive value function computation. The total computational cost grows as $O(2^n \cdot m \cdot C_f)$ where m is the number of samples for expectation estimation and C_f is the cost of one model evaluation.

4.2 Summation Over Permutations

The Shapley value formula requires considering all $n!$ permutations or equivalently all 2^{n-1} coalitions that feature i can join. For the permutation formulation $\phi_i(v) = \frac{1}{n!} \sum_{\pi \in \Pi(N)} [v(S_\pi^i \cup \{i\}) - v(S_\pi^i)]$, exact computation requires $O(n!)$ evaluations. For the coalition formulation $\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(n-|S|-1)!}{n!} [v(S \cup \{i\}) - v(S)]$, we need $O(2^n)$ evaluations. Both formulations become computationally prohibitive for even modest numbers of features (e.g., $n > 20$).

These challenges motivate **approximation methods** including Monte Carlo sampling of permutations, limiting to local neighborhoods of features, and model-specific optimizations that exploit structure in f .

5 Worked Example: Exact Shapley Values for a Boolean Model

To illustrate the Shapley framework in a simple, fully tractable setting, we consider a synthetic example with three binary inputs and a single scalar output. This example serves as a bridge between the classical coalitional game view and the feature attribution view in machine learning.

5.1 Boolean Function with Three Inputs

Let the input space be $\{0, 1\}^3$, and define the Boolean function $f : \{0, 1\}^3 \rightarrow \{0, 1\}$ by

$$f(x_1, x_2, x_3) = (x_1 \oplus x_2) \wedge x_3, \quad (3)$$

where $\oplus$ denotes the XOR (exclusive OR) operator and $\wedge$ denotes the logical AND. Thus $f(x) = 1$ only when

- $x_3 = 1$, and
- exactly one of x_1, x_2 equals 1.

The full truth table is given in Table 2.

x_1	x_2	x_3	$x_1 \oplus x_2$	$f(x_1, x_2, x_3)$
0	0	0	0	0
0	0	1	0	0
0	1	0	1	0
0	1	1	1	1
1	0	0	1	0
1	0	1	1	1
1	1	0	0	0
1	1	1	0	0

Table 2: Truth table for $f(x_1, x_2, x_3) = (x_1 \oplus x_2) \wedge x_3$.

We will interpret the three features (x_1, x_2, x_3) as the players in a coalitional game, and the **value function** $v(S)$ as the model output when only the features in $S \subseteq N = \{1, 2, 3\}$ are “active”, with the remaining features fixed to a baseline.

5.2 Neural Network Approximation

In practice, instead of working directly with a Boolean function f , we often have a learned model f_θ (e.g. a neural network) that approximates f . For concreteness, consider a small multi-layer perceptron (MLP) with one hidden layer:

$$h = \sigma(W_1 x + b_1) \in \mathbb{R}^4, \quad (4)$$

$$\hat{y} = \sigma(w_2^\top h + b_2) \in (0, 1), \quad (5)$$

where $x \in \mathbb{R}^3$ is the input, $W_1 \in \mathbb{R}^{4 \times 3}$ and $w_2 \in \mathbb{R}^4$ are weight parameters, $b_1 \in \mathbb{R}^4$ and $b_2 \in \mathbb{R}$ are biases, and $\sigma(\cdot)$ is the elementwise ReLU activation in the hidden layer and a sigmoid activation at the output. The model is trained on all $2^3 = 8$ input combinations using binary cross-entropy loss so that $\hat{y} \approx f(x_1, x_2, x_3)$ for every point in the truth table.

In the discussion below, we will work with the exact Boolean function f (for clarity of exposition), but the same definitions and computations apply if f is replaced by a trained neural network f_θ .

5.3 Shapley Values for a Single Input

We now compute the exact Shapley values for a single input x^* under a fixed baseline. Let the feature index set be $N = \{1, 2, 3\}$, and define the baseline input $x^{(0)} \in \mathbb{R}^3$ by

$$x^{(0)} = (0, 0, 0), \quad (6)$$

so that all “missing” features are set to zero. For a given input $x^* = (x_1^*, x_2^*, x_3^*)$, we define a coalition-based input

$$x_S^*(i) = \begin{cases} x_i^*, & \text{if } i \in S, \\ 0, & \text{if } i \notin S, \end{cases} \quad (7)$$

and set the value function

$$v(S) = f(x_S^*), \quad S \subseteq N. \quad (8)$$

We are interested in a specific input

$$x^* = (1, 0, 1), \quad (9)$$

for which

$$f(x^*) = f(1, 0, 1) = 1. \quad (10)$$

The baseline prediction is

$$v(\emptyset) = f(0, 0, 0) = 0. \quad (11)$$

Hence the total effect to be distributed by the Shapley values is

$$f(x^*) - f(x^{(0)}) = 1 - 0 = 1. \quad (12)$$

5.3.1 Coalition Values

For each subset $S \subseteq N$, we construct x_S^* by taking the corresponding coordinates from x^* and setting others to zero. The resulting coalition values $v(S)$ are shown in Table 3.

S	x_S^*	$v(S) = f(x_S^*)$
$\emptyset$	$(0, 0, 0)$	0
$\{1\}$	$(1, 0, 0)$	0
$\{2\}$	$(0, 0, 0)$	0
$\{3\}$	$(0, 0, 1)$	0
$\{1, 2\}$	$(1, 0, 0)$	0
$\{1, 3\}$	$(1, 0, 1)$	1
$\{2, 3\}$	$(0, 0, 1)$	0
$\{1, 2, 3\}$	$(1, 0, 1)$	1

Table 3: Coalition-based inputs and values for $x^* = (1, 0, 1)$ under baseline $x^{(0)} = (0, 0, 0)$.

5.3.2 Permutation-Based Shapley Computation

With three features, there are $3! = 6$ permutations of $N = \{1, 2, 3\}$. For each permutation π , we start from the empty coalition and add features in the order specified by π , recording the marginal contribution $v(S \cup \{i\}) - v(S)$ for each feature i when it is added. The Shapley value for feature i is the average of its marginal contributions over all permutations:

$$\phi_i = \frac{1}{3!} \sum_{\pi \in \Pi(N)} (v(S_i^\pi \cup \{i\}) - v(S_i^\pi)), \quad (13)$$

where S_i^π is the set of features that appear before i in permutation π .

We enumerate the permutations and contributions in Table 4. Here $\Delta_i^{(\pi)}$ denotes the marginal contribution of feature i for permutation π .

Permutation π	Contributions $\Delta_1^{(\pi)}$	$\Delta_2^{(\pi)}$	$\Delta_3^{(\pi)}$
$(1, 2, 3)$	0	0	1
$(1, 3, 2)$	0	0	1
$(2, 1, 3)$	0	0	1
$(2, 3, 1)$	1	0	0
$(3, 1, 2)$	1	0	0
$(3, 2, 1)$	1	0	0

Table 4: Marginal contributions $\Delta_i^{(\pi)}$ for each permutation π when explaining $x^* = (1, 0, 1)$.

Summing over all permutations and dividing by $3! = 6$ gives the Shapley

values:

$$\phi_1 = \frac{0 + 0 + 0 + 1 + 1 + 1}{6} = \frac{3}{6} = 0.5, \quad (14)$$

$$\phi_2 = \frac{0 + 0 + 0 + 0 + 0 + 0}{6} = 0, \quad (15)$$

$$\phi_3 = \frac{1 + 1 + 1 + 0 + 0 + 0}{6} = \frac{3}{6} = 0.5. \quad (16)$$

5.3.3 Additivity and Interpretation

The Shapley values for $x^* = (1, 0, 1)$ under baseline $x^{(0)} = (0, 0, 0)$ satisfy the efficiency (additivity) property:

$$f(x^*) - f(x^{(0)}) = 1 - 0 = 1 = \phi_1 + \phi_2 + \phi_3 = 0.5 + 0 + 0.5. \quad (17)$$

Thus the total increase in prediction from the baseline to the observed input is decomposed into equal contributions from features x_1 and x_3 , while feature x_2 receives zero attribution for this particular input. This matches our intuitive understanding of the Boolean function: in the configuration $(1, 0, 1)$, the output is 1 because both x_1 and x_3 are 1 and jointly satisfy the $(x_1 \oplus x_2) \wedge x_3$ condition, whereas $x_2 = 0$ does not contribute positively or negatively relative to the baseline.

6 Dropout and Learning a Background Distribution

Up to this point, we have assumed a deterministic predictor $f : \mathbb{R}^n \rightarrow \mathbb{R}$. However, modern neural networks frequently include stochastic regularisation layers such as *dropout*. Dropout modifies the network during training by randomly removing (or “dropping”) a subset of hidden units. This introduces additional subtleties when interpreting model predictions using Shapley values.

6.1 Dropout as a Stochastic Function Approximator

Given an input x , a dropout network computes

$$f_{\theta,d}(x) = f_{\theta}(x; d), \quad (18)$$

where d denotes the random dropout mask. During training, each unit is independently dropped with probability p :

$$d_j \sim \text{Bernoulli}(1 - p). \quad (19)$$

At test time, the standard deterministic approximation rescales weights, giving

$$\tilde{f}_{\theta}(x) = \mathbb{E}_d[f_{\theta,d}(x)], \quad (20)$$

which corresponds to the “expected network” under dropout noise.

Dropout therefore implicitly trains the network to be robust to missing information. Conceptually, dropout teaches the network that certain units—and therefore certain *features*—may be absent. This has direct implications for Shapley values: removal of a feature during attribution becomes less surprising to a model that learned to operate under missing information.

6.1.1 Dropout and the Value Function

Consider the value function $v(S)$ used in Shapley attribution. With dropout, the natural generalisation is

$$v(S) = \mathbb{E}_d \left[f_\theta(x_S, x_{N \setminus S}^{(0)}; d) \right], \quad (21)$$

where $x^{(0)}$ is a chosen baseline.

Hence Shapley values can be interpreted as:

The expected marginal contribution of feature i when added to a coalition S , averaged both over coalitions and over dropout masks.

This expectation is expensive to compute exactly and is typically approximated by using the deterministic test-time network $\tilde{f}_\theta(x)$.

6.2 When the Model Learns the Background

In classical Shapley feature attribution, the baseline $x^{(0)}$ is fixed externally (e.g. zero vector, mean of dataset, or random samples). However, many neural networks implicitly (or explicitly) learn an internal “background model” of what inputs typically look like.

There are two main mechanisms through which this occurs:

6.2.1 1. Dropout Encourages Modeling of Missing Features

Because dropout randomly removes hidden activations during training, the network must learn to infer missing information from the remaining units. In effect, dropout trains the network to be:

- tolerant to missing signals, and
- able to reconstruct meaningful intermediate representations using typical background patterns.

Thus, when Shapley methods “remove” a feature by substituting a baseline value, the model may behave differently from the case where true input information is absent.

6.2.2 2. Models Often Learn Dataset-Level Means

Even without dropout, a neural network frequently learns the empirical mean of the data distribution. For example, in MNIST, black background pixels dominate the dataset, so the model naturally allocates low activation to regions corresponding to the background.

For a baseline $x^{(0)}$, two important cases arise:

Zero Baseline. For grayscale MNIST, $x^{(0)} = 0$ corresponds to black pixels. Since most MNIST pixels are near 0, the model may implicitly treat 0 as a “background” pattern.

Learned Baseline. Hidden layers may acquire biases that reproduce an approximate background image even when features are removed. This means that using a zero baseline may not truly simulate “removing” a feature; instead it may trigger a learned bias pattern.

6.2.3 3. Explicitly Learning a Background Model

One can also intentionally train a network to reconstruct background inputs. For example:

- train an autoencoder on MNIST backgrounds,
- train a denoising model to fill in missing pixels,
- or add a loss term encouraging $f_\theta(x^{(0)})$ to approximate a learned baseline.

In these settings, the baseline used by Shapley values should ideally match the background distribution the model has internalised.

6.3 Implications for Shapley Feature Attribution

When the model is trained with dropout or implicitly learns a background distribution, Shapley values may change substantially depending on the baseline.

Given a baseline $x^{(0)}$, the additive decomposition

$$f(x) - f(x^{(0)}) = \sum_{i=1}^n \phi_i \quad (22)$$

only faithfully represents the model’s behaviour if $x^{(0)}$ is meaningful to the model.

If the baseline is:

- **in-distribution** (e.g. average MNIST background), Shapley values tend to provide intuitive, robust explanations.

- **out-of-distribution** (e.g. constant zero when the model never sees zeros), Shapley values may be misleading due to unpredictable model extrapolation.

Dropout accentuates this effect because the model has learned to process partially missing representations. Thus the choice of baseline becomes even more critical.

6.4 Summary

- Dropout introduces stochasticity and encourages robustness to missing information.
- Neural networks may implicitly learn a “background” representation of typical inputs.
- Shapley values depend critically on the baseline $x^{(0)}$; incorrect or unrealistic baselines can distort attribution.
- When dropout or background modeling is present, choosing baselines that reflect the data distribution leads to more reliable attributions.

This connection between regularisation, background learning, and Shapley values helps explain why attribution methods behave differently across different architectures and training protocols.